

processes.py Documentation

Process Management and JSON Persistence Module

Updated: 2026-03-01

Warning: This documentation was generated by an AI assistant. However I have reviewed this file and made changes accordingly if needed.

Abstract

`processes.py` is a Python module designed for process management with JSON persistence capabilities. It provides three core classes: `Mindmap` for process relationship mapping, `TimeStamp` for temporal tracking with disruptions, and `Process` for comprehensive process management with automatic instance tracking and $O(1)$ lookup by unique identifier.

Contents

1 Overview

The `processes.py` module implements a process management system with the following key features:

- **Automatic Instance Tracking:** Uses `weakref.WeakSet` to track all created `Process` instances without manual registration
- **O(1) UID Lookup:** Dictionary-based JSON storage enables constant-time process retrieval
- **Temporal Disruption Handling:** `TimeStamp` class supports nested disruptions (pauses, async interruptions)
- **Merge-based Storage:** `storeAll()` preserves existing data while updating changed processes
- **Graph-based Relationships:** `Mindmap` class maps process dependencies and connections

2 Dependencies

The module requires only Python standard library modules:

```
1 import random      # For UID generation
2 import json        # For persistence
3 import weakref     # For automatic instance tracking
4 import os          # For file operations
```

3 Class Reference

3.1 Mindmap

The `Mindmap` class implements a directed graph structure for mapping process relationships.

3.1.1 Constructor

```
1 class Mindmap:
2     def __init__(self, map: dict):
3         """
4         A mindmap can map the schematics of a process.
5         The map dictionary includes nodes (process names)
6         and edges (connections between nodes).
7         """
8         self.map = map
```

3.1.2 Methods

3.1.3 Connection Direction

The `connect()` method supports three connection types via the `direction` parameter:

- `direction=None`: Bidirectional connection (both nodes reference each other)
- `direction=False`: Source to node (`srcNode` → node)
- `direction=True`: Node to source (node → `srcNode`)

Method	Returns	Description
<code>nodes()</code>	<code>dict_keys</code>	Returns all node names in the mindmap
<code>edges()</code>	<code>dict_values</code>	Returns all edge connections
<code>connections(node)</code>	<code>list</code>	Returns nodes connected to given source node
<code>connected(src, node)</code>	<code>bool</code>	Checks if two nodes are connected
<code>connect(src, node, direction)</code>	<code>None</code>	Creates connection with optional directionality

3.2 TimeStamp

The `TimeStamp` class handles temporal tracking with support for nested disruptions.

3.2.1 Constructor

```

1 class TimeStamp:
2     def __init__(self, _init: int, _term: int, _dis=None):
3         """
4         _init: time of initiation (Unix timestamp)
5         _term: time of termination (Unix timestamp, -1 = ongoing)
6         _dis: disruptions dict {"type": TimeStamp}
7         """
8         self.ts = {"initialized": _init, "terminated": _term}
9         self.ts["disruptions"] = []
10
11         if _dis is not None:
12             for i in _dis:
13                 self.ts["disruptions"].append({i: _dis[i]})

```

3.2.2 Properties

The `TimeStamp` class provides property accessors for `_init` and `_term` that map to the internal `ts` dictionary:

```

1 @property
2 def _init(self):
3     return self.ts["initialized"]
4
5 @_init.setter
6 def _init(self, value):
7     self.ts["initialized"] = value
8
9 @property
10 def _term(self):
11     return self.ts["terminated"]
12
13 @_term.setter
14 def _term(self, value):
15     self.ts["terminated"] = value

```

Usage:

```

1 ts = TimeStamp(1000, -1)           # init=1000, term=-1 (ongoing)
2 ts._term = 5000                   # Updates ts["terminated"] = 5000

```

```
3 print(ts.ts["terminated"])      # 5000 (not -1)
```

3.2.3 Instance Methods

Method	Returns	Description
<code>add_disruption(type, ts)</code>	<code>None</code>	Adds a disruption <code>TimeStamp</code> to this timestamp
<code>serialize_disruptions()</code>	<code>list</code>	Converts disruptions to JSON-serializable format
<code>to_hierarchical_dict()</code>	<code>dict</code>	Returns complete timeline dictionary

3.2.4 Static Methods

```
1 @staticmethod
2 def de_serialize(dis: list) -> dict:
3     """
4     Reconstructs TimeStamp objects from JSON disruption list.
5     Returns dict of {disruption_type: TimeStamp}.
6     """
```

3.3 Process

The `Process` class extends `TimeStamp` with process management capabilities and automatic instance tracking.

3.3.1 Class-Level Instance Tracking

```
1 class Process(TimeStamp):
2     __instances = weakref.WeakSet()
3     # Automatically tracks all Process instances
```

The `weakref.WeakSet` ensures:

- Automatic registration in `__init__`
- No memory leaks (weak references allow garbage collection)
- No manual tracking required

3.3.2 Constructor

```
1 def __init__(self, name: str, _init: float, _term: float = -1,
2             layer: int = 0, epoch: int = 0, _dis=None,
3             parent_uid: str|None = None, subtasks: list|None = None):
4     """
5     name: Process name/identifier
6     _init: Start time (Unix timestamp)
7     _term: End time (-1 = ongoing)
8     layer: Hierarchical layer (0 = parallel processing allowed)
9     epoch: Training epoch number (0 = model architecture processes)
10    _dis: Disruptions dictionary
```

```

11 parent_uid: Parent process UID (None = root)
12 subtasks: List of child process UIDs
13
14 Auto-generates UID: "{epoch}x{layer}-{8-digit-hex}"
15 Auto-registers to __instances
16 """

```

3.3.3 Instance Methods

Method	Returns	Description
to_hirearchial_dict()	dict	Full process dictionary with timeline
to_storage_dict()	tuple	(uid, dict) for dict storage
write(ofile)	str	Save/update single process to JSON

3.3.4 Class Methods

Method	Returns	Description
load_dict(data)	Process	Create Process from dictionary
storeAll(ofile)	str	Merge all instances to JSON file
loadAll(ofile)	list	Load all processes from file
load(uid, ofile)	Process None	O(1) lookup by UID

3.3.5 Static Methods

```

1 @staticmethod
2 def get_epoch_from_uid(uid: str) -> int:
3     """Extracts epoch number from UID format 'epochxlayer-hex'"""
4     # Example: "5x2-a3f2b1c9" -> 5
5     # "0x0-abc12345" -> 0
6
7 @staticmethod
8 def get_layer_from_uid(uid: str) -> int:
9     """Extracts layer number from UID format 'epochxlayer-hex'"""
10    # Example: "5x2-a3f2b1c9" -> 2
11    # "0x1-def67890" -> 1
12
13 @staticmethod
14 def remove(uid: str, ofile: str) -> bool:
15     """Remove process by UID from storage"""
16
17 @staticmethod
18 def clear_storage(ofile: str) -> None:
19     """Clear all processes from storage"""

```

4 JSON Storage Format

The module uses a dictionary-based JSON structure for O(1) UID lookup. UIDs follow the format epochxlayer-randomid:

```

1 {
2   "processes": {
3     "0x0-abc12345": {
4       "name": "Transformer_from_scratch.py",
5       "uid": "0x0-abc12345",
6       "parent_uid": null,
7       "epoch": 0,
8       "layer": 0,
9       "timeline": {
10        "initialized": 1234567890.123,
11        "terminated": 1234567900.456,
12        "disruptions": []
13      },
14      "subtasks": ["0x1-def67890"]
15    },
16    "5x2-ghi54321": {
17      "name": "train_epoch_05",
18      "uid": "5x2-ghi54321",
19      "parent_uid": "0x0-abc12345",
20      "epoch": 5,
21      "layer": 2,
22      "timeline": {
23        "initialized": 1234568000.000,
24        "terminated": 1234568100.000,
25        "disruptions": []
26      },
27      "subtasks": []
28    }
29  }
30 }

```

UID Format: epochxlayer-randomid

- **epoch:** Training epoch number (0 for model architecture)
- **layer:** Hierarchical layer (for parallel/sequential grouping)
- **randomid:** 8-character hex string for uniqueness

Key Design Decision: Using UID as dictionary key enables $O(1)$ lookup vs $O(n)$ list iteration.

5 Usage Examples

5.1 Creating and Saving Processes

```

1 from processes import Process, TimeStamp
2
3 # Create architecture processes (epoch=0)
4 encoder_block = Process("EncoderBlock_L0", time.time(), layer=2, epoch=0)
5 ffn_block = Process("FeedForwardBlock_L0", time.time(), layer=2, epoch=0)
6
7 # Create training epoch processes (epoch > 0)
8 epoch_process = Process("train_epoch_05", time.time(), layer=0, epoch=5)

```

```
9
10 # Set termination time (properly updates timestamp)
11 encoder_block._term = time.time()
12
13 # Add disruption
14 pause = TimeStamp(50, 75)
15 encoder_block.add_disruption("pause", pause)
16
17 # Save single process
18 encoder_block.write("processes.json")
19
20 # Save all tracked instances
21 Process.storeAll("processes.json")
```

5.2 Loading Processes

```
1 # O(1) lookup by UID
2 p = Process.load("5x2-abc12345", "processes.json")
3 if p:
4     print(f"Found: {p.name}, Epoch: {p.epoch}, Layer: {p.layer}")
5
6 # Load all processes
7 all_processes = Process.loadAll("processes.json")
8
9 # Load from dictionary (for custom processing)
10 data = {"name": "Test", "uid": "0x1-1234abcd",
11         "epoch": 0, "layer": 1,
12         "timeline": {"initialized": 0, "terminated": 100}}
13 p = Process.load_dict(data)
```

5.3 Parsing UIDs

```
1 # Parse epoch and layer from UID
2 epoch = Process.get_epoch_from_uid("5x2-abc12345") # Returns 5
3 layer = Process.get_layer_from_uid("5x2-abc12345") # Returns 2
4
5 # Architecture processes (epoch 0)
6 arch_epoch = Process.get_epoch_from_uid("0x0-def67890") # Returns 0
7 arch_layer = Process.get_layer_from_uid("0x0-def67890") # Returns 0
```

5.4 Removing Processes

```
1 # Remove single process
2 success = Process.remove("1-a3f2", "processes.json")
3
4 # Clear all storage
5 Process.clear_storage("processes.json")
```

5.5 Using Mindmap for Dependencies

```
1 from processes import Mindmap
2
3 # Create process dependency map
```

```
4 mindmap = Mindmap({
5     "DataLoading": [],
6     "Preprocessing": ["DataLoading"],
7     "Training": ["Preprocessing"],
8     "Validation": ["Training"]
9 })
10
11 # Check dependencies
12 print(mindmap.connections("Training")) # ["Preprocessing"]
13 print(mindmap.connected("DataLoading", "Training")) # False
```

6 Method Reference by Category

6.1 Serialization

- `TimeStamp.serialize_disruptions()` → JSON-serializable disruption list
- `TimeStamp.to_hierarchical_dict()` → Complete timeline dict
- `Process.to_hierarchical_dict()` → Complete process dict
- `Process.to_storage_dict()` → (uid, dict) tuple

6.2 Deserialization

- `TimeStamp.de_serialize(list)` → {type: `TimeStamp`} dict
- `Process.load_dict(dict)` → `Process` instance

6.3 Persistence

- `Process.write(ofile)` → Save/update single process
- `Process.storeAll(ofile)` → Merge all instances to file
- `Process.loadAll(ofile)` → Load all as list
- `Process.load(uid, ofile)` → O(1) lookup
- `Process.remove(uid, ofile)` → Remove by UID
- `Process.clear_storage(ofile)` → Clear all

7 Notes and Limitations

- **Thread Safety:** File operations are not atomic; concurrent writes may cause data loss
- **Memory Management:** `WeakSet` auto-removes unreferenced instances from tracking
- **UID Collisions:** 8-digit hex provides 4.3 billion combinations; collision probability negligible
- **JSON Limitations:** All timestamps stored as floats (Unix epoch with microseconds)
- **Disruption Nesting:** Disruptions can contain disruptions recursively
- **Epoch Tracking:** Architecture processes use epoch=0; training epochs use actual epoch number
- **TimeStamp Properties:** `_init` and `_term` properties ensure proper dictionary updates